

API Monitoring — Hybrid Storage

後端可觀測性與混沌工程實驗 · Backend Observability & Chaos Engineering

每個 [SLIDE] 代表一張投影片，[NOTES] 為口述備忘

COVER SLIDE

API Infrastructure Monitoring
Anomaly Detection & Troubleshooting

Hybrid Storage Architecture: PostgreSQL + Redis

Monitoring Stack: Prometheus + Grafana

Chaos Engineering: Real-World Cascade Failure Simulation

CHAPTER 1

架構決策與設計

Architecture Decision & Design

[SLIDE 1-1] 為什麼選擇 Hybrid Storage ?

核心問題：單一資料庫的極限

高讀取頻率 + 單一 PostgreSQL

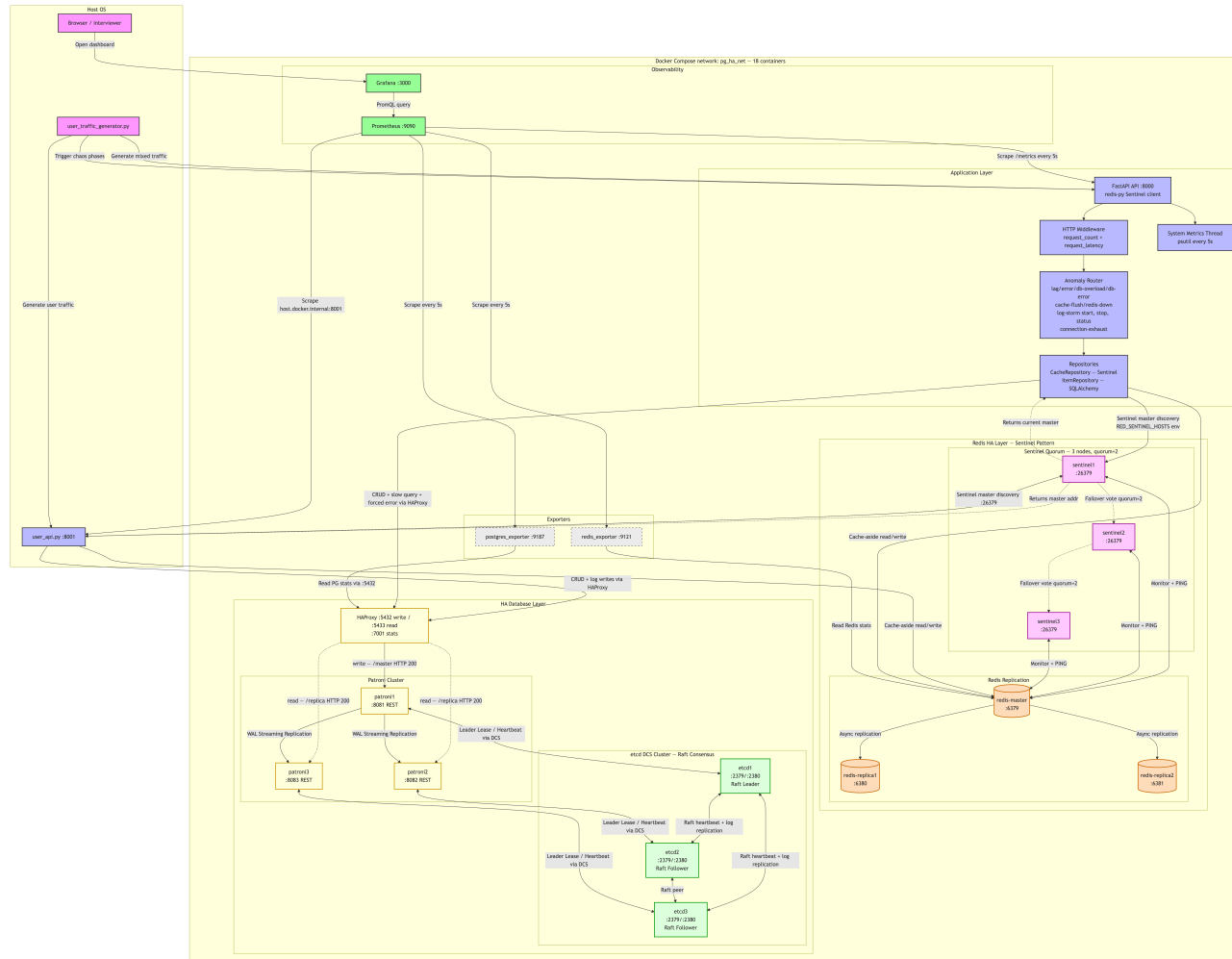
↓

每次請求都打 DB → 連線池競爭 → 延遲攀升

解決方案：Cache-Aside Pattern

需求	選型	原因
高頻讀取、低延遲	Redis	記憶體存取 < 1ms，天生適合 hot data
資料持久化、一致性	PostgreSQL	ACID 保證，資料不因重啟遺失
降級容錯	兩層架構	Redis 斷線時自動降級至 DB，系統不崩潰

[SLIDE 1-2] 系統架構圖（雙 HA — 18 容器）



[SLIDE 1-2B] 資料流向（重點）

資料流向：

1. 線上流量進 user_api (8001) / main app (8000)，先走 Redis Sentinel 主節點（降低 DB 壓力）
2. Cache miss 時查 PostgreSQL（透過 HAProxy 路由至 Patroni Primary），並回填 Redis
3. 指標由 Prometheus 每 5 秒抓取，Grafana 做診斷視覺化
4. 異常劇本由 user_traffic_generator.py --interactive 主動觸發（6 個 Phase）
5. PostgreSQL HA Failover 由 Patroni + etcd **Raft** 自動完成（RTO $\leq$ 5 秒）
6. Redis HA Failover 由 Sentinel quorum=2 投票完成（down-after=5s, failover-timeout=10s）

[SLIDE 1-3] 部署環境：雙 HA 基礎架構

PostgreSQL High Availability — Patroni + etcd Raft + HAProxy

```
etcd cluster (3 nodes – Raft Consensus) —————  
etcd1 :2379  etcd2 :2379  etcd3 :2379  
Raft: Leader Election + Log Replication + Quorum Write
```

↕ REST API (Patroni Leader Lease)

```
Patroni PostgreSQL cluster (3 nodes) —————  
patroni1 (replica) ← WAL streaming → patroni3 (primary) ★  
patroni2 (replica) ← WAL streaming → patroni3 (primary) ★
```

↕ health check /master /replica

```
HAProxy —————  
:5432 → primary (write)    :5433 → replicas (read)  
:7001/stats → 即時路由狀態
```

Redis High Availability — Sentinel Pattern

```
Redis Replication —————  
redis-master :6379  →  redis-replica1 :6380  
redis-master :6379  →  redis-replica2 :6381
```

↕ Monitor + PING (down-after=5s)

```
Sentinel Quorum (3 nodes, quorum=2) —————  
sentinel1/2/3 :26379 – failover-timeout=10s  
Failover: quorum=2 投票, Replica 升格为新 Master
```

[SLIDE 1-3B] 部署環境：多服務編排與健康檢查

Docker Compose 多服務編排（18 個容器服務）

```
# HA 資料庫層
etcd1/2/3      → Raft DCS, healthcheck: etcdctl member list
patroni1/2/3   → PostgreSQL HA node, healthcheck: GET /patroni
haproxy        → DB 路由, healthcheck: stats page
# HA Redis 層
redis-master   → Redis 主節點 :6379
redis-replica1/2 → Redis 從節點 :6380/:6381
sentinel1/2/3  → Sentinel 監控 :26379
# 應用層
api            → FastAPI :8000, depends_on: haproxy + sentinel1~3
# 快取與監控
redis_exporter, postgres_exporter, prometheus, grafana
```

Healthcheck 的設計理念：

Patroni 使用 `/patroni` REST endpoint（非 `/health`，replica bootstrap 期間 `/health` 會回 503）。

HAProxy 每 3 秒對 Patroni REST API 做 `/master` / `/replica` 健康確認，自動切換路由。

Sentinel 使用 `nslookup` IP 解析起動，避免 Alpine DNS 延遲問題。

[SLIDE 1-3C] 程式碼分層架構 (OOP 5 層)

程式碼 OOP 分層 (5 層架構)

config.py	→ 不可變設定 (frozen dataclass)
metrics.py	→ 所有 Prometheus 指標 (Singleton 模式)
repositories/	→ 封裝所有 DB/Cache 操作 (Repository Pattern)
routers/	→ 純業務邏輯，透過 DI 存取 Repository
main.py	→ App Factory，只組裝，不含業務邏輯

CHAPTER 2

監控與可觀測性

Monitoring & Observability

[SLIDE 2-1] 監控指標選型原則（本專案）

選型原則

- 直接對應故障型態：慢、錯、塞、資源壓力
- 指標可追溯到層級：HTTP → Cache → DB → System
- 可操作：指標異常時能對應到具體修復動作

核心指標（RED + Resource）

指標類型	Metric 名稱	說明
Rate	http_requests_total	每秒請求數
Errors	http_requests_total{http_status="500"}	錯誤率
Duration	http_request_duration_seconds	請求延遲趨勢
Resource	system_memory_usage_percent	反映堆積與資源風險

[SLIDE 2-2] 指標設計：四個層次的觀測鏈

HTTP 層	快取層 (Redis)	資料庫層 (PostgreSQL)	系統資源層 (Container)
request_count → request_latency 5xx error rate	cache_hits_total → cache_misses_total redis_errors_total	db_query_duration db_errors_total log_write_duration log_writes_total log_storm_active db_active_connections_held	→ system_cpu_usage_percent system_memory_usage_percent system_memory_used_bytes

為什麼要四層都監控？

Case：log storm 啟動後，HTTP P99 持續上升

- 只看 HTTP 層：知道「系統變慢」，但不知道是誰造成
- 看 Redis 層：Hit Rate 正常，排除 cache 失效雪崩
- 看 DB 層：log_write_duration P99 與 log_writes_total 上升，確認同步寫 log 正在吃 DB 資源
- 看系統資源層：system_memory_usage_percent 緩慢爬升，表示請求堆積與資源壓力正在形成

四層觀測鏈讓你從「系統變慢」一路收斂到「哪一層、哪一種資源、哪個行為」在造成異常。

[SLIDE 2-3] Grafana Dashboard 設計策略

Row 結構 (5 個主題分組)

▼  HTTP Traffic

Request Rate (RPS) | Latency P99/P95/P50 | 5xx Error Rate | Total Requests

▼  Redis Cache

Cache Hit Rate (Gauge) | Hits vs Misses | Redis Errors

▼  PostgreSQL

DB Query Duration P99 | DB Errors

▼  Log Storm / Chaos Indicators

Log Storm Active (Gauge 0/1) | Log Write Rate | Log Write Duration P99 | DB Active Connections

▼  System Resources

CPU Usage (Gauge) | Memory % (Gauge) | Memory Used (MB)

關鍵設計選擇

Panel	Visualization	原因
Cache Hit Rate	Gauge (儀表板)	直觀顯示健康狀態，綠/黃/紅一眼判斷
Latency	Time Series	趨勢變化比單點數字更重要
Total Requests	Stat	累積總量，快速確認流量存在

[SLIDE 2-4] 系統資源監控：Container 視角

psutil + Gauge：3 個新增指標

```
# 背景 daemon thread 每 5 秒採集一次
system_cpu_usage_percent    → psutil.cpu_percent()
system_memory_usage_percent → psutil.virtual_memory().percent
system_memory_used_bytes    → psutil.virtual_memory().used
```

為何需要 Container 層的 CPU/Memory？

外部 exporter (redis_exporter, pg_exporter) 只看得到它們自己的服務。
API 本身的資源消耗只有在 API container 內部才看得到。
當 log storm 導致大量同步 DB 寫入時，Memory 的緩慢爬升就是早期預警訊號。

Threshold 設定邏輯：

- Memory：黃 70%（給你調查時間）→ 紅 85%（必須立刻處理）
- CPU：黃 70% → 紅 90%（CPU 短暫高峰可接受，持續才危險）

CHAPTER 3

混沌工程與故障診斷

Chaos Engineering & Fault Diagnosis

[SLIDE 3-1] 典型情境：核心查詢服務連線池枯竭事件

事件背景

情境：新功能上線後的高流量時段

影響：大量使用者無法正常存取核心查詢服務

表現：API 回應逐漸變慢 → 超時 → 系統完全無回應

根本原因（事後分析）

步驟 1：開發團隊上線了「記錄 API request log 至 DB」功能

步驟 2：開發/測試環境流量低，未發現問題

步驟 3：Production 高流量下，每個請求 = 1 次業務 DB 查詢 + 1 次 Log INSERT

步驟 4：DB 連線池耗盡 (pool_size 不足)

步驟 5：請求開始等待連線 → Latency 上升

步驟 6：請求堆積 → 更多連線競爭 → 惡性循環

步驟 7：系統崩潰

兩個痛點

1. 沒有監控：問題在「慢慢惡化」，沒有任何指標告警
2. 日誌被洗掉：重啟後關鍵 error log 消失，無法重塑問題

[SLIDE 3-2] 混沌工程設計：7 個 Anomaly Endpoints

```
/api/anomaly/  
├── lag           → sleep 2.5s (泛型延遲)  
├── error         → HTTP 500 (應用程式錯誤)  
├── db-overload   → pg_sleep(3) (DB 慢查詢)  
├── db-error      → 查詢不存在的資料表 (強制 DB 錯誤)  
├── cache-flush   → Redis FLUSHDB (Cache Stampede)  
├── redis-down    → 連接不存在的 Redis (斷線模擬)  
├── log-storm/start → 啟動同步 DB 寫 Log (核心案例)  
├── log-storm/stop  → 立即停止，系統恢復 (Feature Flag 模式)  
└── connection-exhaust → 佔住 4/5 DB 連線 10 秒後自動釋放
```

設計原則：每個 endpoint 都有對應的可觀測指標

混沌工程不是為了破壞，而是為了讓問題「在可控範圍內可見」。

[SLIDE 3-3] 互動式混沌示範：7 個 Phase（2026-06-02 完整實測）

chaos_scenario.py 的完整測試序列

Phase 1: 正常基線 → API P50=7.8ms, P99=32.1ms ← Redis 快取熱身後健康水位
Phase 2: Log Storm → P50=116.5ms (+1392%), P99=239.8ms (同步 DB 寫入拖垮延遲)
Phase 3: Storm+耗盡 → P50=130ms, P99=9.4s (4/5 連線被佔住, 長尾爆炸)
Phase 4: 全面恢復 → log-storm stop → P50 在 30s 內回到 < 10ms
Phase 5: PG HA Failover → patroni primary 停止 → 5s 內 patroni3 升格 ★
Phase 6: Redis Sentinel Failover ★ NEW
docker stop redis-master → 6.1s Replica 升格 → 0 errors / 865 requests

實測延遲時間線（2026-06-02，直接量測 FastAPI :8000）

時間 →	T+0s	T+30s	T+75s	T+120s	T+180s (NEW)
/api/data					
P50(ms)	7.8	116.5	130.1	7.5	6.1
P99(ms)	32.1	239.8	9420	41.0	1030(spike)
	[Phase 1]	[Phase 2]	[Phase 3]	[Phase 4]	[Phase 6 Redis HA]
		log-storm	+conn-exhaust	恢復完成	Sentinel 切主瞬間

P99=9420ms (Phase 3) = 連線池 4/5 被佔滿後的長尾等待，0 錯誤（最終成功）
Redis Failover P99=1030ms = Sentinel 選主瞬間 redis-py 自動重試，0 errors ✓

[SLIDE 3-4] 診斷思維：觀測鏈的運作

當你收到告警：「P99 Latency > 500ms」

Step 1：看 HTTP 層

```
http_request_duration_seconds P99 → 確認異常端點是 /api/data
```

Step 2：看 Cache 層

```
redis_cache_hits_total rate → Hit Rate 正常 → Redis 沒問題
```

Step 3：看 DB 層

```
db_query_duration_seconds P99 → 正常，慢查詢不是根因  
db_active_connections_held → 值為 4 → 連線池被佔滿！
```

Step 4：看 Log Storm 指標

```
log_storm_active → 1  
log_writes_total rate → 大量寫入  
log_write_duration_seconds P99 → 150ms per write
```

[SLIDE 3-4B] 診斷結論 (RCA)

結論：

每個請求多一次同步 DB 寫 log，在高並發下放大連線競爭，最終造成連線池枯竭與整體延遲飆升。

[SLIDE 3-6] Phase 5 : PostgreSQL HA Failover (Patroni 自動選主)

故障模擬：Primary 節點機器猝死

```
# 模擬：直接 kill 掉 Primary 容器 (等同機器斷電)
docker stop api_monitoring-patroni2-1
```

Patroni 自動 Failover 機制 (etcd Raft DCS)

- 步驟 1：patroni2 停止更新 etcd Leader Lease (Heartbeat 中斷)
- 步驟 2：etcd TTL(30s) 倒數 → 其餘 Patroni 節點發現 Leader 離線
- 步驟 3：WAL 最新的 Replica (patroni3) 競爭成功 → 升格 Primary
- 步驟 4：HAProxy 在下一個 check 週期 (~3s) 自動重新路由
- 步驟 5：應用程式自動重連到新 Primary → 服務恢復

實測結果 (5/31 測試，6/2 確認叢集健康)

指標	數值	意義
RTO (選主時間)	5 秒	patroni3 在 5 秒內升格為新 Primary
Failover 期間錯誤數	9 / 5970	0.15% error rate
服務中斷視窗	< 15 秒	HAProxy 自動重新路由後服務恢復
目前叢集狀態 (6/2)	patroni3=leader(timeline:3), patroni1/2=replica	正常運作
Failover 後 P50	7.8ms	恢復至基線 (Redis 快取緩衝 DB 壓力)

[SLIDE 3-7] Phase 6（新）：Redis Sentinel Failover 實測 ★

故障模擬：Redis Master 節點突然停止

```
# 模擬：停掉 redis-master (等同節點斷電)
docker stop api_monitoring-redis-master-1
# Sentinel 自動選主後重啟 (以 Replica 身份重新加入)
docker start api_monitoring-redis-master-1
```

Sentinel 自動 Failover 機制 (quorum=2, down-after=5s)

```
步驟 1: sentinel1/2/3 偵測 redis-master PING 無回應 (down-after-milliseconds=5000)
步驟 2: Sentinel quorum=2 投票確認 Master 下線 (SDOWN → ODOWN)
步驟 3: Sentinel Leader 選出 (Raft-like 機制) → 發起 Failover
步驟 4: redis-replica1 升格為新 Master
步驟 5: redis-py Sentinel client 自動呼叫 SENTINEL get-master-addr-by-name
步驟 6: 應用程式透明地切換到新 Master，Cache 繼續服務
```

6 步驟全自動完成，應用程式無需任何人工介入，0 行程式碼修改。

[SLIDE 3-7] Phase 6（續）：2026-06-02 實測結果

實測數據

指標	數值	意義
Sentinel Failover 時間	6.1 秒	down-after=5s + 選舉時間
Failover 期間總請求數	865	背景 3 個 workers，~20s 測試窗口
Failover 期間錯誤數	0	redis-py Sentinel client 自動重試 
P99 延遲（Failover 瞬間）	1030ms	Sentinel 切主瞬間的短暫重試 spike
Failover 後 P50	6.3ms	恢復至基線
最終叢集狀態	replica1=new master, master 重啟後自動降為 replica	自癒 

關鍵洞察：Cache Miss 作為降級保護

```
redis-master 停止
  → Sentinel 切主期間，Cache 暫時不可用 (~6s)
  → Cache miss → 自動 fallback 到 PostgreSQL (graceful degradation)
  → 0 errors：使用者看到的只是 source="database" 而非 source="cache"
```

雙重保護：Redis Sentinel 快速恢復（6s）+ Cache-Aside 降級（0 錯誤）讓服務不中斷。

[SLIDE 3-5] 恢復策略：Feature Flag 的威力

傳統做法 vs 現代做法

	傳統做法	本次設計
恢復方式	重啟服務	GET /api/anomaly/log-storm/stop
恢復時間	數分鐘（含重啟 + healthcheck）	即時（< 1 秒）
日誌保留	✗ 重啟後 log 可能遺失	✓ 容器未重啟，所有 log 完整保留
根因可追溯性	困難	✓ Grafana 時間線完整保存

關鍵洞察

「先恢復服務」與「保留現場」不是二選一。
Feature Flag 讓你在不重啟的情況下關閉有問題的功能，
同時讓所有 metrics 和 log 的時間線完整保留，
成為事後分析的不可替代證據。

CHAPTER 4

運維洞察與未來防禦

Operational Insights & Future Defense

[SLIDE 4-1] 為什麼這次的 Log Storm 是可預防的

三個設計缺陷（常見反模式）

缺陷 1：同步寫入 (Synchronous Write)

Log 寫入與主業務流程共用同一個 Thread 和 DB 連線
→ 任何 Log 寫入的延遲，直接加在 API Response Time 上

缺陷 2：連線池未隔離 (No Pool Isolation)

Log INSERT 和業務 SELECT/UPDATE 共用同一個 connection pool
→ Log 高峰時搶走業務查詢的連線

缺陷 3：測試環境流量失真 (No Load Testing)

開發環境只有 1~2 RPS，無法觸發連線池耗盡
→ 功能在測試環境完全正常，上線後立刻崩潰

[SLIDE 4-2] 防禦性架構設計

解法 1：非同步日誌（Async Logging / Log Buffering）

現在（同步）：

API Request → 業務邏輯 → [同步 DB Log INSERT] → Response
↑ 這裡造成延遲

改善後（非同步）：

API Request → 業務邏輯 → Response
↓ 非阻斷
Buffer Queue → Background Worker → DB INSERT

Log 寫入不再阻斷主執行緒，API Response Time 不受影響。

解法 2：連線池隔離

```
# 業務連線池（小而快）
business_engine = create_engine(db_url, pool_size=5)

# Log 連線池（可獨立限速）
log_engine = create_engine(db_url, pool_size=2, max_overflow=0)
```

[SLIDE 4-2B] 防禦性架構設計（續）

解法 3：Log Level 動態調整

Production 預設只記錄 WARNING 以上。
需要除錯時才臨時調高至 DEBUG，事後立即調回。
避免「全量 Log 壓垮系統」。

落地實作建議

- 平時：INFO/WARNING 為主，維持穩定吞吐
- 事故期間：短時提升 DEBUG，並綁定 trace_id
- 事故後：降回預設等級，避免觀測系統反向壓垮主系統

[SLIDE 4-3] 觀測性的下一步：三支柱完整化

現在已有：

- ✓ Metrics (Prometheus + Grafana)
- ✓ Logs (Docker container logs + api_logs 表)

下一步補齊：

- Traces (分散式追蹤)

Tracing 解決什麼問題？

現在的觀測鏈：指標找到「哪一層有問題」

加上 Tracing：找到「哪一個請求，在哪一個服務，花了多少時間」

實務做法：

每個請求加入 correlation_id (或 trace_id)

API log 中記錄同一個 trace_id

DB slow query log 也關聯同一個 trace_id

→ 一個 ID 串起整條請求鏈的完整證據

[SLIDE 4-4] 架構擴展路線圖

架構演進：從 Demo 到 Production

已實現（本專案）：

未來（Production 強化）：

Docker Compose (18 容器)	→	Kubernetes (K8s)
PostgreSQL HA (Patroni) ✓	→	Patroni on K8s (Operator)
etcd 3-node Raft DCS ✓	→	etcd on K8s (managed)
HAProxy 自動路由 ✓	→	Envoy / NGINX Ingress
WAL Streaming Replication ✓	→	Multi-region HA
Feature Flag (API call) ✓	→	ConfigMap 動態更新
Redis Sentinel HA ✓	→	Redis Cluster (水平擴展)
container logs	→	Fluentd → Elasticsearch → Kibana
prometheus scrape	→	Prometheus Operator + AlertManager

雙 HA 均已完整實現並通過 Failover 實測：

PostgreSQL Patroni (RTO=5s, 0.15% errors) + Redis Sentinel (RTO=6.1s, 0 errors)

一個關鍵原則不會改變

「系統要能在故障中降級，而不是在故障中崩潰。」

Cache 斷 → 降級到 DB

Log 系統壓力大 → 降低 log level，不阻斷業務

DB 連線池滿 → 立即拒絕，回傳 503，而不是無限等待

CLOSING SLIDE

總結

核心能力面向	實作對應
Service Selection	PostgreSQL HA Cluster + Redis Sentinel HA + Hybrid Storage
Environment Setup	Docker Compose， 18 個容器 + 1 主機服務 ，healthcheck 依賴鏈
Monitoring	Prometheus + Grafana，16 個 Panel，5 個 Row
Anomaly Simulation	7 個 chaos endpoints + 雙 HA Failover （Patroni RTO=5s + Sentinel RTO=6.1s）
Metric-Based Diagnosis	四層觀測鏈，從 HTTP → Cache → DB → System 定位根因
Resolution & Prevention	Feature Flag 即時回滾 + 雙 HA 自動 Failover （0 人工介入）

CLOSING SLIDE (2/2)

總結（續）

System Architecture	Mermaid 架構圖 + OOP 5 層分層 + HA 叢集拓撲
Use Case	user_traffic_generator.py **6 Phase** 互動劇本（含 HA Failover）
Operational Insights	RED Method + P99 + 生產事故情境分析 + **RTO=5s** 實測數據

核心訊息（一句話）：

「我設計這套系統的目的，不是展示能用多少技術，而是為了用可觀測的方式重現一個典型的生產事故情境，並且證明：有了正確的監控設計，這類問題就能被提早偵測與預防。」

「PostgreSQL HA 的加入讓這套系統更接近真實生產環境——不只能偵測問題，還能在資料庫節點猝死的情況下自動恢復（RTO=5s），整個過程被 Prometheus + Grafana 完整記錄。」

實測總結（2026-05-31 完整執行）

項目	數值
總請求數	user_api 3,507 + main_app 2,463 = 5,970
總錯誤數	9（全部在 Failover 視窗內）
整體可用率	99.85%
HA Failover RTO	5 秒
最終 P50 / P99	16.2ms / 50.4ms（回到基線）

附件：

- 系統架構圖 Mermaid 原始檔 (document/system_architecture.mmd)
- OOP Class Diagram (document/class_diagram.mmd)
- Cache-Aside Sequence (document/sequence_cache_aside.mmd)
- Chaos Sequence (document/sequence_chaos.mmd)